CE152: Object-Oriented Programming

Arrays

Dr. Sebastian Halder

CSEE

2026

Arrays

What are arrays?

- ***Arrays are extremely important!***
- What defines array is that they are a sequence of values of the same type
- For example we may create an array of String to store the days of the week:

index	value
0	"MONDAY"
1	"TUESDAY"
2	"WEDNESDAY"
3	"THURSDAY"
4	"FRIDAY"
5	"SATURDAY"
6	"SUNDAY"

Size and indices

- The size of arrays is fixed which means we cannot change the number of elements after creating it
  - We can change the values of individual elements
  - We can also just use a part of the array (this would be memory inefficient but faster than creating a new array)
- In Java we can get the size of an array x using x.length
  - Remember that the *indices start at 0* and run to (x.length - 1)!
- There are also other data structures in Java that we can use such as lists which we will learn about later

Arrays

- Arrays are indicated by the []
- We can make all kinds of arrays out of any *Object* or *basic data type*. E.g.:

```
int[] intArray = new int[5];
double[] doubleArray = new double[100];
Lecturer[] lecturerArray = new Lecturer[5000];
```

- The number in the [] indicates the number of elements in the array (e.g. the int array has 5 elements)
- Initialising them is a bit tricky because they have a fixed size

Java array creation (1)

```java
// declare the array
String[] days;
// initialise the array
days = new String[7];

// assign values
days[0] = "MONDAY";
days[1] = "TUESDAY";
days[2] = "WEDNESDAY";
days[3] = "THURSDAY";
days[4] = "FRIDAY";
days[5] = "SATURDAY";
days[6] = "SUNDAY";
```

Java array creation (2)

```
// Declare and intialise in the same line
String[] days = new String[7];

// Declare, initialise and assign
String[] days = {"MONDAY", "TUESDAY", "WEDNESDAY",
"THURSDAY", "FRIDAY", "SATURDAY", "SUNDAY"};
```

Default values and access

- When we initialise an array but do not assign values *default values* are assigned
- In cases of int[] it would be filled with zeros, String[] is filled with empty `null`
- Accessing an array with an index that does not exist causes a runtime exception:

```
System.out.println(days[days.length]);
```

```
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException:
Index 7 out of bounds for length 7
  at mainLecture4.main(mainLecture4.java:17)
```

Printing to command line

- Remember if you need to convert the Array to String *before* printing:

```
System.out.println(days);
// Converts the Object type and hashcode to String
System.out.println(days.toString());
```

```
[Ljava.lang.String;@38af3868
[Ljava.lang.String;@38af3868
```

- Convert the contents to String:

```
System.out.println(Arrays.toString(days));
```

```
[MONDAY, TUESDAY, WEDNESDAY, THURSDAY,
  FRIDAY, SATURDAY, SUNDAY]
```

Field descriptors

- Strings such as `[Ljava.lang.String;@38af3868` are called field descriptors
- The `[` indicates an array, the `L` that a class name will follow, followed by the hash code that identifies the object For further information see:
  - https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-4.html

Arrays needs to be imported

- For this we use the keyword *import* at the very beginning of our source file

```
import java.util.Arrays;
```

- This allows us to refer to a class directly instead of using the fully qualified name
- We can write:

```
Arrays.toString(days);
```

- Instead of:

```
java.util.Arrays.toString(days);
```

Comparing Arrays

```
int[] myNumbers1 = new int[] {1,2,3};
int[] myNumbers2 = {1,2,3};
```

- This comparison will yield false:

```
System.out.println(myNumbers1.equals(myNumbers2));
```

```
false
```

- This is because the method compared the hashcodes of the Objects which differ. We need to compare them using a static method in `Arrays`:

```
System.out.println(Arrays.equals(myNumbers1, myNumbers2));
```

```
true
```

Arithmetic with arrays

```
// declaring and initialising
int[] a = new int[10];

// assigning values
for (int i = 0; i < a.length; i++)
    a[i] = (int) (100 * Math.random());
```

Iterating over all elements

- We used for loops to iterate over all elements of the array
- We can quickly create such a loop using "for each":

```
// This will iterate over all elements in a
for(int elem: a) System.out.println(elem);
```

- Note that we do not need indices
- Assigning a new value to elem does not change the value in the Array

```
// This will have no effect
for(int elem: a) elem = 42;
```

Sorting an array

- Often we will want to sort the values in our Arrays
- For this we can use sort() from java.utils.Arrays

```java
int[] a = new int[10];
for (int i = 0; i < a.length; i++)
    a[i] = (int) (100 * Math.random());

System.out.println(Arrays.toString(a));
Arrays.sort(a);
System.out.println(Arrays.toString(a));
```

```
[23, 70, 29, 20, 89, 75, 97, 14, 13, 35]
[13, 14, 20, 23, 29, 35, 70, 75, 89, 97]
```

Arrays are collections of addresses in memory

- We have not talked about what arrays actually "are"
- In fact they are not their content, they are *addresses in memory*
  - This means if we copy an array to another variable and change it, *both variables will be affected* because they point to the same space in memory
- Arrays are more like aliases or shortcuts point to a particular place

```
int[] orig = {3, 1, 4, 1, 5};
System.out.println(orig[0]);
int[] origAsWell = orig;
origAsWell[0] = -1;
System.out.println(orig[0]);
```

- Output:

```
3
-1
```

So can we copy arrays?

- Yes! But we need to create the target array first and then copy the *dereferenced* elements

```
int[] copy = new int[orig.length];
for(int i = 0; i<orig.length;i++){
  copy[i] = orig[i]; }

// This will not change orig
copy[0] = 5;
```

- Of course this already exists as a static function which is much more efficient than our loop and is how we should copy arrays:

```
int[] arrayCopy = Arrays.copyOf(orig, orig.length);
System.out.println(Arrays.toString(arrayCopy));
```

Where do Arrays go when they become inaccessible?

- It can happen that we lose access to an array:

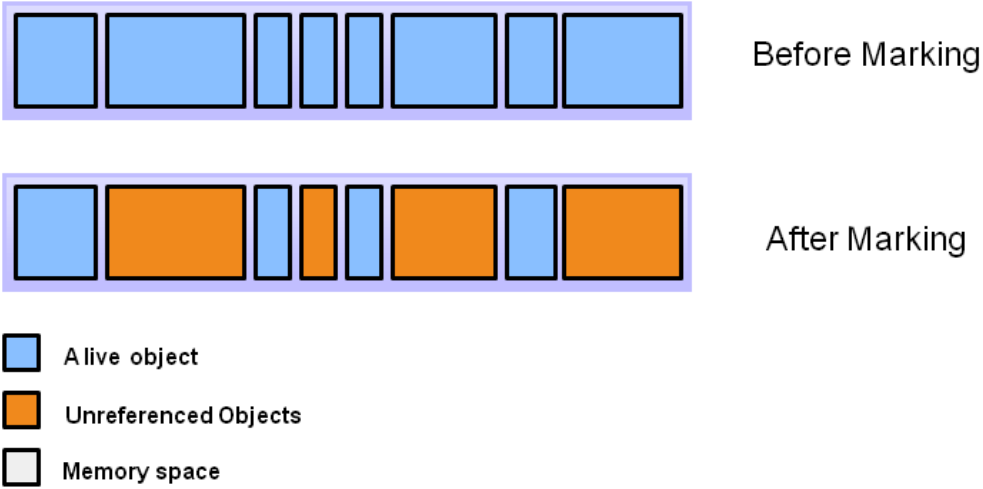
```
int[] a = {3, 1};
int[] b = {4, 1, 5, 9};
a = b;
```

- Now both a and b point to the memory space created when we initialised b
- The memory space a pointed to after initialisation has become inaccessible
- Is this bad? What could happen if it is not taken care of?

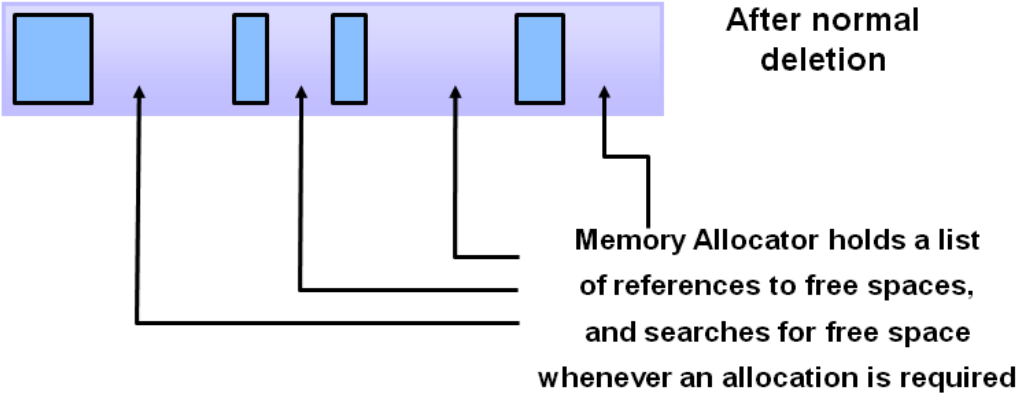
The heap

- The JVM stores Arrays and Objects in a memory area called the *heap*
- While the program is running the JVM will check which items on the *heap* are still accessible
- If an item is no longer accessible the JVM marks this area as free and deletes the original item
- Sometimes used areas of the heap are compacted
- This process is called *garbage collection*
- Automatic garbage collection causes a bit of overhead but prevents *memory leaks* (these can occur e.g. in C++)

Garbage collection - Marking

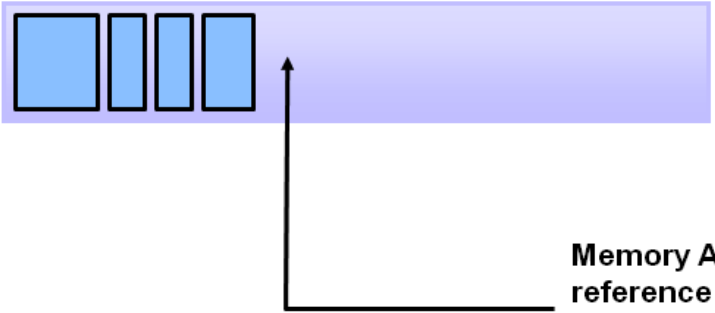


Garbage collection - Deleting



See Oracle Java tutorial

Garbage collection - Compacting



After normal Deletion with compacting

Memory Allocator holds the reference to the beginning of free space, and allocated memory sequentially then on.

See Oracle Java tutorial

Multidimensional arrays

Why multidimensional arrays?

- So far we have only used one dimensional arrays
- There are situations in which we will want to use multidimensional arrays
  - pixels on a screen
  - pairwise distances
  - matrices

How do we create multidimensional arrays?

- To create a multidimensional array we need to create an array of arrays

```
int[][] a = {{1,2,3},{4,5,6}};
```

- This array will have a length of 2
- Each element will have a length of 3

Example of filling and accessing a 2D Array

```
int rows = 5;
int cols = 5;
double[][] randomArray = new double[rows][cols];
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        randomArray[i][j] = Math.random();

for (double[] row : randomArray)
    System.out.println(Arrays.toString(row));
```

Summary Arrays

- Arrays are a basic concept that exists in most programming languages
- Array programmes usually involve for loops iterating over the indexes or directly over array elements (for-each)
- Arrays can hold large amounts of data efficiently
- Java's garbage collector will reclaim unused heap space
- We can implement multidimensional arrays using arrays-of-arrays
- We can use arrays to perform vector or matrix multiplications (see videos)
- *Matrix operations are extremely important* in many science and engineering fields as well as in *3D graphics for video games!*

Command line arguments

The secret behind String[] args

- There is actually a reason why we provide a parameter String[] args in:

```
public static void main(String[] args){}
```

- The reason is that any parameters we pass when calling this function from the command line will be stored in this array
- If we pass several parameters they will be separate items in the array

Example String[] args usage

```java
public class RandomIntCmdLine {
    public static void main(String[] args) {
        int n = Integer.parseInt(args[0]);
        double r1 = Math.random();
        int r2 = (int) (r1 * n);
        System.out.println("random integer is " + r2);
    }
}
```

- To run this from the command line we compile it and switch to the out/Production folder if we are using IntelliJ
- In any case the folder should contain "*.class" files
- Then we can enter:

```
$ java RandomIntCmdLine 4
  random integer is 2
```

Summary

Summary

- We can use a for-each loop to iterate over all items of an array
- Multidimensional arrays can be created using multiple []
  - e.g. int[][] will create a 2D array of ints
- Arrays and other global variables will be stored on the heap
- Function variables will be stored on the stack
- Primitives are passed as their value
- Objects are passed as a reference
  - So we can manipulate the original variable in the function which is called a side-effect

Thank you!